



AUTO AI

Car Discovery & Outreach Agent

Document Type	Business Requirements Document (BRD)
Version	1.0
Date	April 19, 2026
Author	Neeraj Nagpal
Status	Active Development
Classification	Confidential

Powered By: LangChain · OpenAI GPT-4o-mini · Marketcheck · Twilio · SendGrid · Streamlit

Table of Contents

#	Section Title
1	Executive Summary
2	Business Objectives
3	Scope
4	Functional Requirements
5	Non-Functional Requirements
6	System Architecture
7	Technology Stack
8	Data Models
9	External API Summary
10	Environment Configuration
11	Agent Pipeline Flow
12	Known Limitations
13	Future Enhancements
14	Setup & Run Instructions
15	PR / FAQ — Press Release & Frequently Asked Questions
16	Product Strategy
17	Design Architecture Diagram
18	Detailed Workflow
19	Agent Deep Dive

SECTION 1

Executive Summary

AUTO AI is an AI-powered car discovery and outreach platform built to automate the end-to-end process of finding, ranking, and communicating personalized used car listings to buyers. The system leverages a multi-agent pipeline — combining real-time marketplace inventory data, AI-based scoring, and automated email/SMS delivery — to eliminate the manual effort of browsing multiple car listing sites.

AUTO AI reduces the car buying research process from hours to seconds by combining real-time dealer inventory data with AI-driven personalization and automated outreach.

SECTION 2

Business Objectives

#	Objective	Priority
1	Automate used car discovery based on buyer preferences	High
2	Surface real, live inventory from dealer websites via Marketcheck API	High
3	Rank listings by relevance using a multi-factor scoring algorithm	High
4	Deliver personalized results to buyers via Email and/or SMS	Medium
5	Provide graceful fallback to AI-simulated listings when live data is unavailable	Low

SECTION 3

Scope

In Scope

- Web-based user interface for entering car preferences (Streamlit)
- Real-time car inventory search via Marketcheck API
- AI-powered make/model name normalization (handles user misspellings)
- Multi-factor listing ranking engine (price, mileage, color match)
- Automated email delivery via SendGrid with GPT-generated content
- Automated SMS delivery via Twilio with GPT-generated content
- Graceful fallback to GPT-simulated listings if live API is unavailable

Out of Scope

- User account management / authentication / login
- Payment or financing integration
- Vehicle history reports (Carfax, AutoCheck)
- Direct dealer contact or lead form submission
- Mobile native app (iOS / Android)
- AutoTrader and CarGurus direct API (no public API available)

SECTION 4

Functional Requirements

4.1 User Input — Search Preferences

Field	Type	Required	Notes
Make	Text	Yes	Auto-corrected via GPT (handles misspellings)
Model	Text	Yes	Auto-corrected via GPT
Min Price (\$)	Number	Yes	USD — must be less than Max Price
Max Price (\$)	Number	Yes	USD
Exterior Color	Dropdown	No	Any, White, Black, Silver, Gray, Red, Blue, Green, Other
Interior Color	Dropdown	No	Any, Black, Beige, Gray, Brown, White, Red, Other
Max Mileage	Number	No	Odometer cap in miles
Location	Text	Yes	ZIP code or City, State format
Search Radius	Slider	Yes	10 – 200 miles
Email Delivery	Checkbox	No	Requires valid email address
SMS Delivery	Checkbox	No	Requires valid phone number (+1 format)

4.2 Search Agent

- Accepts user preferences as structured Pydantic model (CarPreferences)
- Normalizes make/model spelling using GPT-4o-mini before API call
- Queries Marketcheck API (GET /v2/search/car/active) with full filter set
- Parses response: title, price, mileage, year, colors, dealer, location, URL, source domain
- Falls back to GPT-simulated listings if Marketcheck key is absent or API fails
- Surfaces a warning banner in the UI when fallback mode is triggered

4.3 Ranking Agent — Scoring Formula

Factor	Max Points	Scoring Logic
Price Proximity	40 pts	Closer to midpoint of budget range = higher score
Mileage	30 pts	Lower mileage relative to max = higher score
Exterior Color Match	15 pts	Substring match between preference and listing
Interior Color Match	15 pts	Substring match between preference and listing
TOTAL	100 pts	Top 5 listings returned, sorted descending

4.4 Outreach Agent

Channel	Generator	Delivery Service	Content
Email	GPT-4o-mini	SendGrid API	Personalized HTML email with listing cards, prices, links
SMS	GPT-4o-mini	Twilio API	Concise summary ≤300 chars — top 3 cars with price & mileage

SECTION 5

Non-Functional Requirements

Category	Requirement
Performance	Marketcheck API call must complete within 15 seconds
Availability	Runs locally via Streamlit; no uptime SLA currently
Security	API keys stored in .env file — never committed to version control
Scalability	Stateless pipeline; each search is fully independent
Resilience	Full GPT fallback if Marketcheck API fails or returns 0 results
Compatibility	Python 3.9+ on macOS / Linux

SECTION 6

System Architecture

LAYER	COMPONENT	TECHNOLOGY	ROLE
Frontend	app.py	Streamlit	User interface — preferences sidebar & results grid
Orchestration	orchestrator.py	Python	Coordinates Search → Rank → Outreach pipeline

Agent	search_agent.py	LangChain + Requests	Make/model normalization, Marketcheck query, GPT fallback
Agent	ranking_agent.py	Python	Multi-factor scoring and top-5 selection
Agent	outreach_agent.py	LangChain + APIs	Email/SMS content generation and delivery
Data Models	models.py	Pydantic v2	CarPreferences and CarListing schema validation
Config	config.py	python-dotenv	Environment variable management
External API	Marketcheck	REST API v2	Real-time used car inventory from dealer websites
External API	OpenAI GPT-4o-mini	OpenAI SDK	Normalization, email, SMS generation, fallback listings
External API	SendGrid	SendGrid SDK	Transactional HTML email delivery
External API	Twilio	Twilio SDK	SMS message delivery

Pipeline Flow

1	User Input Enter preferences → click Find Cars
2	Search Agent Normalize make/model → Query Marketcheck → Parse listings
3	Ranking Agent Score by price + mileage + color → Return top 5
4	Outreach Agent Generate email/SMS via GPT → Deliver via SendGrid/Twilio
5	Display Results Show listing cards with source badge, score, and live links

SECTION 7

Technology Stack

Layer	Technology	Version	Purpose
Frontend	Streamlit	≥1.32.0	Web UI
Agent Framework	LangChain	≥0.2.0	LLM chain orchestration
LLM	OpenAI GPT-4o-mini	Latest	Normalization, content generation, fallback
LLM Client	langchain-openai	≥0.1.0	OpenAI API integration
Data Validation	Pydantic	≥2.0.0	CarPreferences & CarListing models

Car Inventory	Marketcheck API	v2	Real-time used car listings
Email Delivery	SendGrid	≥6.10.0	Transactional email
SMS Delivery	Twilio	≥9.0.0	SMS messaging
HTTP Client	Requests	≥2.31.0	Marketcheck API calls
Config	python-dotenv	≥1.0.0	Environment variable management
Language	Python	3.9+	Runtime

SECTION 8

Data Models

CarPreferences

Field	Type	Required	Description
make	str	Yes	Vehicle make (e.g. Porsche) — auto-corrected
model	str	Yes	Vehicle model (e.g. 911) — auto-corrected
price_min	int	Yes	Minimum budget in USD
price_max	int	Yes	Maximum budget in USD
exterior_color	str?	No	Preferred exterior color
interior_color	str?	No	Preferred interior color
max_mileage	int?	No	Maximum odometer reading in miles
location	str	Yes	ZIP code or City, State
radius_miles	int	Yes	Search radius in miles
delivery_email	bool	Default T	Send results via email
delivery_sms	bool	Default F	Send results via SMS
user_email	str?	No	Recipient email address
user_phone	str?	No	Recipient phone number

CarListing

Field	Type	Description
title	str	Listing headline (e.g. 2019 Porsche 911 Turbo S)
price	int	Listed price in USD

mileage	int	Odometer reading in miles
year	int	Model year
exterior_color	str?	Exterior color
interior_color	str?	Interior color
dealer_name	str?	Selling dealer name
location	str?	Dealer city and state
listing_url	str?	Direct link to the listing page
match_score	float?	Computed relevance score (0–100)
source	str?	Origin domain (e.g. dealer website)

SECTION 9

External API Summary

API	Status	Base URL	Auth Method	Primary Use
Marketcheck	Active	api.marketcheck.com/v2	api_key param	Car inventory search
OpenAI	Active	api.openai.com/v1	Bearer token	LLM — normalize, generate, fallback
SendGrid	Active	api.sendgrid.com/v3	API key header	Email delivery
Twilio	Active	api.twilio.com	AccountSID + Token	SMS delivery
AutoDev	Reserved	api.auto.dev	Bearer token	Future VIN enrichment
AutoTrader	N/A	No public API	Enterprise only	Not integrated
CarGurus	N/A	No inventory search endpoint	Contact required	Not integrated
Cars.com	N/A	Internal API only	Not available	Not integrated

SECTION 10

Environment Configuration

Variable	Required	Description
OPENAI_API_KEY	Yes	OpenAI API key for GPT-4o-mini
MARKETCHECK_API_KEY	Recommended	Marketcheck key – enables real listings
AUTODEV_API_KEY	No	AutoDev key — reserved for future VIN enrichment

TWILIO_ACCOUNT_SID	If SMS	Twilio account SID
TWILIO_AUTH_TOKEN	If SMS	Twilio auth token
TWILIO_PHONE_NUMBER	If SMS	Twilio outbound phone number
SENDGRID_API_KEY	If Email	SendGrid API key
SENDGRID_FROM_EMAIL	If Email	Verified SendGrid sender email

SECTION 11

Agent Pipeline Flow

Step	Agent / Component	Input	Output	Fallback
1	Input Validation	User form data	CarPreferences model	Validation error shown
2	Search Agent	CarPreferences	List of CarListing objects	GPT-simulated listings
3	Ranking Agent	List[CarListing]	Top 5 scored listings	None needed
4	Outreach Agent	Top 5 + user contacts	Email/SMS delivery result	Error shown in UI
5	Display	Ranked listings + status	Result cards + badges	Warning banner

SECTION 12

Known Limitations

Limitation	Detail
AutoTrader / CarGurus data	Not available — both block third-party scraping; no public APIs exist
Marketcheck price filter	API may return listings slightly outside price range; ranking agent re-prioritizes
Free tier API limits	Marketcheck free tier has monthly call volume limits
Location input	Works best with ZIP code; city/state parsing may be less precise
No persistent storage	Search results are not saved between sessions
Python 3.9 compatibility	str None union syntax not supported — uses Optional[] instead

SECTION 13

Future Enhancements

Priority	Enhancement	Benefit
High	VIN-based vehicle history enrichment via AutoDev API	Reliability data per listing
High	Save search history and results to SQLite database	Persistent user sessions
Medium	Display photo thumbnails from Marketcheck media links	Richer listing cards
Medium	User accounts with saved preferences and price alerts	Repeat engagement
Medium	Price trend analysis vs. market value	Buyer negotiation insight
Low	Deploy to cloud (Streamlit Cloud / AWS / GCP)	Public accessibility
Low	Mobile-responsive UI improvements	Better mobile experience
Low	Support for new/CPO inventory	Broader inventory coverage

SECTION 14

Setup & Run Instructions

Prerequisites

- Python 3.9+
- OpenAI API key (required)
- Marketcheck API key (recommended for real listings)

Installation

```
git clone
cd AUTO_AI
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
# Edit .env and add your API keys
```

Running the App

```
streamlit run app.py
# Opens automatically at http://localhost:8501
```

SECTION 15

PR / FAQ — Press Release & Frequently Asked Questions

Press Release

FOR IMMEDIATE RELEASE

AUTO AI Eliminates the Pain of Car Shopping — AI Agent Finds, Ranks, and Delivers Your Perfect Car in Seconds

Anaheim, CA — April 19, 2026 — Today, AUTO AI launches an intelligent car discovery platform that automates the entire used car search process from start to finish. Powered by LangChain, OpenAI GPT-4o-mini, and the Marketcheck inventory API, AUTO AI finds real dealer listings, scores them against your exact preferences, and delivers personalized results directly to your inbox or phone — all in under 30 seconds.

The Problem: Buying a used car today means spending hours on AutoTrader, CarGurus, and dealer websites — manually filtering, comparing, and still not knowing if you found the best deal. The process is fragmented, time-consuming, and frustrating.

The Solution: AUTO AI acts as your personal car-buying agent. Enter your make, model, budget, color, mileage tolerance, and location. The AI pipeline takes over — searching real inventory, ranking every listing with a transparent 100-point score, and sending you a curated shortlist via email or SMS. Every 'View Listing' link goes directly to the dealer's page.

Built for: Car buyers who value their time, automotive dealerships building buyer engagement tools, and developers exploring multi-agent AI pipelines for real-world applications.

Frequently Asked Questions (FAQ)

Q: Is the car data real or simulated?

A: When a Marketcheck API key is configured, all listings are real and sourced directly from dealer websites. Without a key, the system falls back to GPT-simulated listings clearly marked with an 'AI Simulated' orange badge.

Q: Why don't I see AutoTrader or CarGurus listings?

A: Neither platform offers a public API. AutoTrader requires an enterprise partnership (typically \$1,000+/month). CarGurus has no inventory search endpoint. Marketcheck is the best publicly available aggregator that covers real dealer inventory.

Q: How accurate is the match score?

A: The score is transparent and rule-based: 40pts for price fit, 30pts for mileage, 15pts each for exterior and interior color. It is not an ML model — it is deterministic and fully auditable.

Q: Can I use this without Twilio or SendGrid?

A: Yes. Both are optional. If you don't configure them, results are still displayed in the browser UI. Email and SMS are only sent when the keys are present and the user opts in.

Q: What happens if the Marketcheck API fails?

A: The system automatically falls back to GPT-simulated listings and shows a yellow warning banner in the UI explaining the fallback. No error is thrown to the user.

Q: Can the app handle misspelled car makes?

A: Yes. A GPT-4o-mini normalization step corrects make/model spelling before hitting the Marketcheck API — so 'porche 911' becomes 'Porsche 911' automatically.

Q: Is my data stored anywhere?

A: No. The application is stateless. No search history, preferences, or results are persisted between sessions. All data exists only in memory during a single session.

SECTION 16

Product Strategy

Vision



Strategic Pillars

Real Data First	Connect to live dealer inventory via Marketcheck. AI simulation is a fallback, never the default goal.
Transparency	Every match score is explainable — buyers see exactly why a car ranked #1 vs #5.
Zero Friction UX	One form, one click. The agent pipeline handles all complexity invisibly behind the scenes.
Omnichannel Delivery	Results reach buyers where they are — browser, email, or SMS — without requiring an account or login.
Extensible by Design	New data sources (VIN history, photos, pricing trends) can plug in as additional agents without rearchitecting.

Competitive Positioning

Capability	AUTO AI	AutoTrader	CarGurus	Manual Search
Real inventory data	Yes	Yes	Yes	Yes
Public API access	Yes	No	No	N/A
AI-based ranking	Yes	No	Partial	No
Automated email/SMS	Yes	No	No	No
Make/model autocorrect	Yes	No	No	No
Free to use	Yes	No	No	Yes
Customizable pipeline	Yes	No	No	No

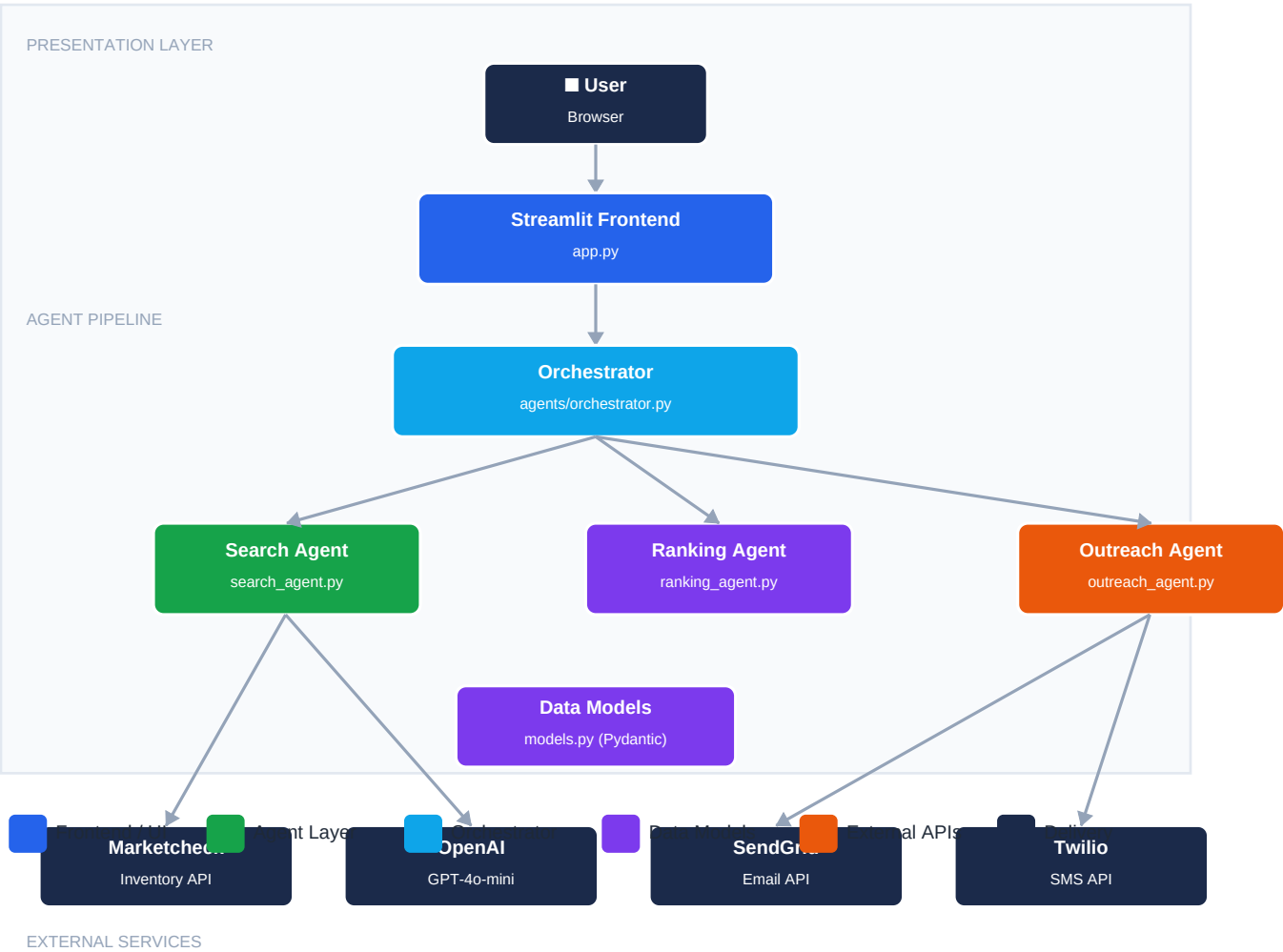
Go-to-Market Phases

Phase	Name	Focus	Target
1	MVP (Now)	Local Streamlit app, Marketcheck integration	Developer / personal use
2	Beta	Cloud deploy, user accounts, saved searches	Early adopters / dealerships
3	Growth	Photo thumbnails, VIN history, price alerts	Consumer car buyers
4	Scale	White-label API for dealers, CRM integrations	Enterprise / dealer networks

SECTION 17

Design Architecture Diagram

The diagram below shows the full system architecture of AUTO AI, from user interaction through the agent pipeline to external APIs and delivery channels.



SECTION 18

Detailed Workflow

The workflow below traces the full lifecycle of a single car search request through the AUTO AI system — from user input to delivered results.



Step-by-Step Breakdown

Step 1	User Input & Validation User fills the Streamlit sidebar with make, model, budget, mileage, color, location, radius, and delivery preferences. The app validates all required fields and checks that price_min < price_max before proceeding.
Step 2	Orchestrator Kicks Off Pipeline run_pipeline() is called with a validated CarPreferences Pydantic model. The orchestrator calls each agent in sequence: Search → Rank → Outreach, passing results between agents and collecting a final results dictionary.
Step 3	Search Agent — Normalize GPT-4o-mini receives the raw make/model strings and returns the corrected official names as a JSON object. Example: 'porche' → 'Porsche'. This prevents zero-result API calls caused by typos.
Step 4	Search Agent — Query Marketcheck The agent calls GET /v2/search/car/active with all filters: make, model, price range, mileage cap, exterior color, ZIP/city, and radius. Up to 10 results are returned sorted by price ascending. Each listing is parsed into a CarListing Pydantic object.
Step 5	Search Agent — Fallback If MARKETCHECK_API_KEY is not set, or if the API returns 0 results, or if an exception is raised, the agent falls back to GPT-4o-mini to simulate 5 realistic listings. A warning flag is set and passed through the pipeline.

Step 6	Ranking Agent — Score & Sort Each CarListing is scored on 4 factors (price 40pts, mileage 30pts, ext color 15pts, int color 15pts). Listings are sorted descending by score. The top 5 are returned.
Step 7	Outreach Agent — Generate Content If email is enabled: GPT-4o-mini writes a personalized HTML email body with listing cards, prices, mileage, colors, dealer info, and 'View Listing' links. If SMS is enabled: GPT writes a ≤300-character summary of the top 3 listings.
Step 8	Outreach Agent — Deliver SendGrid sends the HTML email to the user's address. Twilio sends the SMS to the user's phone number. Both return a success/failure dict that is surfaced in the UI.
Step 9	Display Results The Streamlit UI renders 5 listing cards in a 3-column grid. Each card shows: title, price, year, mileage, colors, dealer, location, match score, source badge (green = real data, orange = AI simulated), and a clickable 'View Listing →' link.

SECTION 19

Agent Deep Dive

AUTO AI is built on a three-agent architecture coordinated by a central orchestrator. Each agent has a single, well-defined responsibility and communicates via typed Pydantic models.

[agents/search_agent.py](#)

Search Agent		agents/search_agent.py
Role:	Discover real car listings from live inventory APIs and normalize user input.	
Input	CarPreferences (make, model, price range, mileage, colors, location, radius)	
Output	list[CarListing] + optional warning string	
Tools	• OpenAI GPT-4o-mini (normalization) • Marketcheck REST API v2 • GPT-4o-mini (fallback simulation)	
Steps	1. Call GPT-4o-mini to correct make/model spelling 2. Parse location string into ZIP or city/state params 3. Call Marketcheck GET /v2/search/car/active with all filters 4. Parse JSON response into CarListing objects with source domain 5. If 0 results or exception → call GPT fallback and set warning flag	
Fallback	GPT-4o-mini generates 5 simulated listings marked 'AI Simulated'	

[agents/ranking_agent.py](#)

Ranking Agent		agents/ranking_agent.py
Role:	Score and prioritize listings by relevance to buyer preferences.	
Input	CarPreferences + list[CarListing]	
Output	list[CarListing] — top 5, sorted by match_score descending	
Tools	• Pure Python — no external API or LLM calls	
Steps	1. Price proximity: score up to 40pts based on distance from budget midpoint 2. Mileage: score up to 30pts — lower mileage relative to cap = higher score 3. Exterior color: +15pts flat if preference substring matches listing color 4. Interior color: +15pts flat if preference substring matches listing color 5. Sort all listings descending by total score, return top 5	

Fallback	No fallback needed — pure deterministic algorithm	
		agents/outreach_agent.py
Outreach Agent		agents/outreach_agent.py
Role:	Generate personalized content and deliver results via email and SMS.	
Input	CarPreferences + top 5 CarListings	
Output	dict with email/SMS success status and generated content	
Tools	• OpenAI GPT-4o-mini (email + SMS writing) • SendGrid API (email) • Twilio API (SMS)	
Steps	1. If email enabled: GPT writes personalized HTML email body with listing cards 2. Send HTML email via SendGrid to user's address with subject line 3. If SMS enabled: GPT writes ≤300 char SMS summary of top 3 listings 4. Send SMS via Twilio from configured phone number to user's number 5. Return success/failure dict for each channel back to orchestrator	
Fallback	Delivery failure is caught and shown as error in UI — pipeline continues	